Raspberry Pi & Julia

Guide to Tinkering with Hardware and Code

Abdelbacet Mhamdi

2026-06-25

MT @ ISET Bizerte

Outline

1. General Overview

Why Use Raspberry Pi

Raspberry Pi is a popular choice for beginners because it is affordable, well-documented, and has a large community. It ships with a familiar Debian-based environment, making it easy to get started with Linux. It is equally suited for advanced users who want to build custom hardware projects, run home servers, or experiment with electronics via its 40-pin GPIO header.

What is Raspberry Pi

Raspberry Pi is a **low-cost** operating systems. It is a popular platform for single-board computer *capable of running* Linux-based embedded systems, IoT projects, home automation, education, and prototyping. It was originally developed by the Raspberry Pi Foundation (now Raspberry Pi Ltd.) with the goal of promoting computer science education.

The official operating system is **Raspberry Pi OS** (formerly called Raspbian until May 2020). It is based on the Debian Linux distribution and uses the `apt` (Advanced Package Tool) package manager.

Connectivity

Testing and Setting Up Wi-Fi

1.1.1 Checking Connectivity

Before configuring network access, you should connect the raspberry pi to your machine using ethernet and verify that the Raspberry Pi is reachable from your local machine:

```bash
ping raspberrypi.local
```

If the ping succeeds, the Pi is on the network and responding to mDNS (multicast DNS) queries. If it fails, check the physical Ethernet connection.

INFO

i

Create a new connection profile of type Ethernet on the physical interface enp4s0. The PC will act as a DHCP server and automatically assign an IP address to any device connected via Ethernet (the Pi), while also enabling NAT to share the laptop's internet connection.

```bash
nmcli connection add type ethernet ifname enp4s0 con-name "rpi-share" ipv4.method shared
```

Bash

Activate and inspect the connection

```bash
nmcli connection up "rpi-share"
nmcli connection show "rpi-share"
```

Bash

Perform a ping sweep across all 254 addresses, without port scanning

```bash
nmap -sn 10.42.0.0/24
```

Bash

1.1.2 Wi-Fi Configuration

On the Raspberry Pi, use `nmcli` (NetworkManager command-line interface) to configure Wi-Fi connections.

List available Wi-Fi networks to check that the raspberry pi can detect your network:

```bash
nmcli dev wifi list
```

Connect to a Wi-Fi network:

```bash
sudo nmcli dev wifi connect "SSID_NAME" password "WIFI_PASSWORD"
```

Replace `SSID_NAME` with the network name and `WIFI_PASSWORD` with the password. The `sudo` prefix is required because Wi-Fi configuration is a privileged operation.

WARNING

⚠️ Make sure your machine is connected to the same network as the Pi. Once Wi-Fi is configured, you can remove the Ethernet cable while staying connected to the Raspberry Pi terminal. If you disconnect before this step, you will lose connectivity and must reconnect via Ethernet to fix the Wi-Fi configuration.

1.1.3 Packages Installation

Before installing any software, it is good practice to refresh the local package index and then apply all available upgrades:

```bash
sudo apt update  # refresh the package list
sudo apt full-upgrade  # upgrade packages, handling dependency changes
```

`full-upgrade` is preferred over `upgrade` because it correctly handles cases where packages need to be added or removed to satisfy updated dependencies.

To install a specific package:

```bash
sudo apt install <package_name>
```

SSH

Secure Shell Configuration

SSH (Secure Shell) is a cryptographic network protocol for secure remote access to a machine over an unsecured network. On Raspberry Pi OS, the `openssh-server` package is already installed but **disabled by default** for security reasons.

There are three common ways to enable it:

Option 1 — raspi-config (recommended for interactive use):

```bash
sudo raspi-config
# Navigate to: Interface Options → SSH → Enable
```

1.2 SSH Setup

Option 2 — systemctl (enable on a running system):

```bash
sudo systemctl enable ssh  # enable SSH to start on boot
sudo systemctl start ssh  # start the SSH service immediately
```

Option 3 — headless setup (before first boot):

Create an empty file named `ssh` (no extension) in the `/boot` or `/bootfs` partition of the SD card. The OS will detect it on first boot, enable the SSH service, and delete the file.

Once SSH is enabled, connect from another machine with:

```bash
ssh <username>@<hostname>.local
# e.g. ssh pi@raspberrypi.local
```

1.2.1 SSH Key Authentication

Connecting with a password is convenient but less secure. The recommended approach is **public-key authentication**: you generate a key pair on your client machine, then place the public key on the Raspberry Pi. From that point on, no password is needed and brute-force attacks become ineffective.

1.2.2 How it works

A key pair consists of two files:

Private key (`~/.ssh/rpi`) stays on your machine, never shared.
Public key (`~/.ssh/rpi.pub`) copied to the Pi. It can only be used to verify someone who holds the matching private key.

1.2.3 Generate a key pair (on your local machine)

Ed25519 is the current recommended algorithm: it is compact, fast, and more secure than RSA.

```bash
ssh-keygen -t ed25519 -C "your_email@example.com"
```

Bash

You will be prompted for:

File location press Enter to accept the default (`~/.ssh/rpi`).

Passphrase strongly recommended. It encrypts the private key on disk so that even if the file is stolen, it cannot be used without the passphrase.

The command produces two files:

```
~/.ssh/rpi       # private key — keep this secret
~/.ssh/rpi.pub   # public key  — this goes on the Pi
```

1.2.4 Copy the public key to the Raspberry Pi

The simplest method is `ssh-copy-id`, which handles directory creation and permissions automatically:

```bash
ssh-copy-id <username>@<pi-ip-address>
# e.g. ssh-copy-id pi@10.42.0.39
```

Bash

You will be asked for your Pi password one final time. After this step, the public key is appended to `~/.ssh/authorized_keys` on the Pi.

1.2.5 Connect using the key

```bash
ssh <username>@<pi-ip-address>
# e.g. ssh pi@192.168.1.123
```

SSH automatically finds and uses `~/.ssh/id_ed25519`. If you set a passphrase, you will be prompted for it once. To avoid re-entering it every session, add the key to the SSH agent:

```bash
eval "$(ssh-agent -s)"   # start the agent
ssh-add ~/.ssh/rpi  # load the key (prompts for passphrase once)
```

INFO

SSH Client Configuration File

The `~/.ssh/config` file lets you define **host aliases** with all connection parameters, so you never have to remember long ssh commands again.

```
Host rpi
	HostName raspberrypi.local
	User pi
	IdentityFile ~/.ssh/rpi
	ServerAliveInterval 60
```

With this configuration in `~/.ssh/config`, you can simply run:

```bash
ssh rpi
```

Bash

1.2 SSH Setup

Once key authentication is confirmed to be working, you can disable password logins entirely to harden the Pi against brute-force attacks. Edit the SSH server config:

```bash
sudo vim /etc/ssh/sshd_config
```

Find and set the following lines (*restart the ssh service afterwards*):

```
PasswordAuthentication no
PubkeyAuthentication yes
```

Then restart the SSH service.

> **WARNING**
>
> ⚠️ Do not disable password login until you have verified that key authentication works in a separate terminal session. Locking yourself out would require physical access to the Pi to recover.

Git and GitHub

Version Control and Collaboration

1.3 Git and GitHub

Git is a distributed version control system that tracks changes to files and directories over time. It allows you to commit snapshots of your work, branch off for experiments, and merge changes back together.

GitHub is a cloud-based hosting platform for Git repositories. It adds collaboration features such as pull requests, issues, and Actions on top of standard Git.

To install Git:

```bash
sudo apt install git
```

Bash

Basic initial configuration:

```bash
git config --global user.name  "Your Name"
git config --global user.email "you@example.com"
```

Bash

To clone an existing repository from GitHub:

```bash
git clone https://github.com/a-mhamdi/rpi-onramp.git
```

Bash

Using a key with GitHub

The same key pair `rpi` and `rpi.pub` can authenticate you to GitHub, removing the need for HTTPS tokens when pushing code.

Display your public key and copy the output. Then go to `GitHub` → `Settings` → `SSH and GPG keys` → `New SSH key`, paste it in, and save. Test the connection:

```bash
ssh -T git@github.com
# Expected: Hi <username>! You've successfully authenticated...
```

To switch an existing local repository from HTTPS to SSH:

```bash
git remote set-url origin git@github.com:<user>/<repo>.git
```

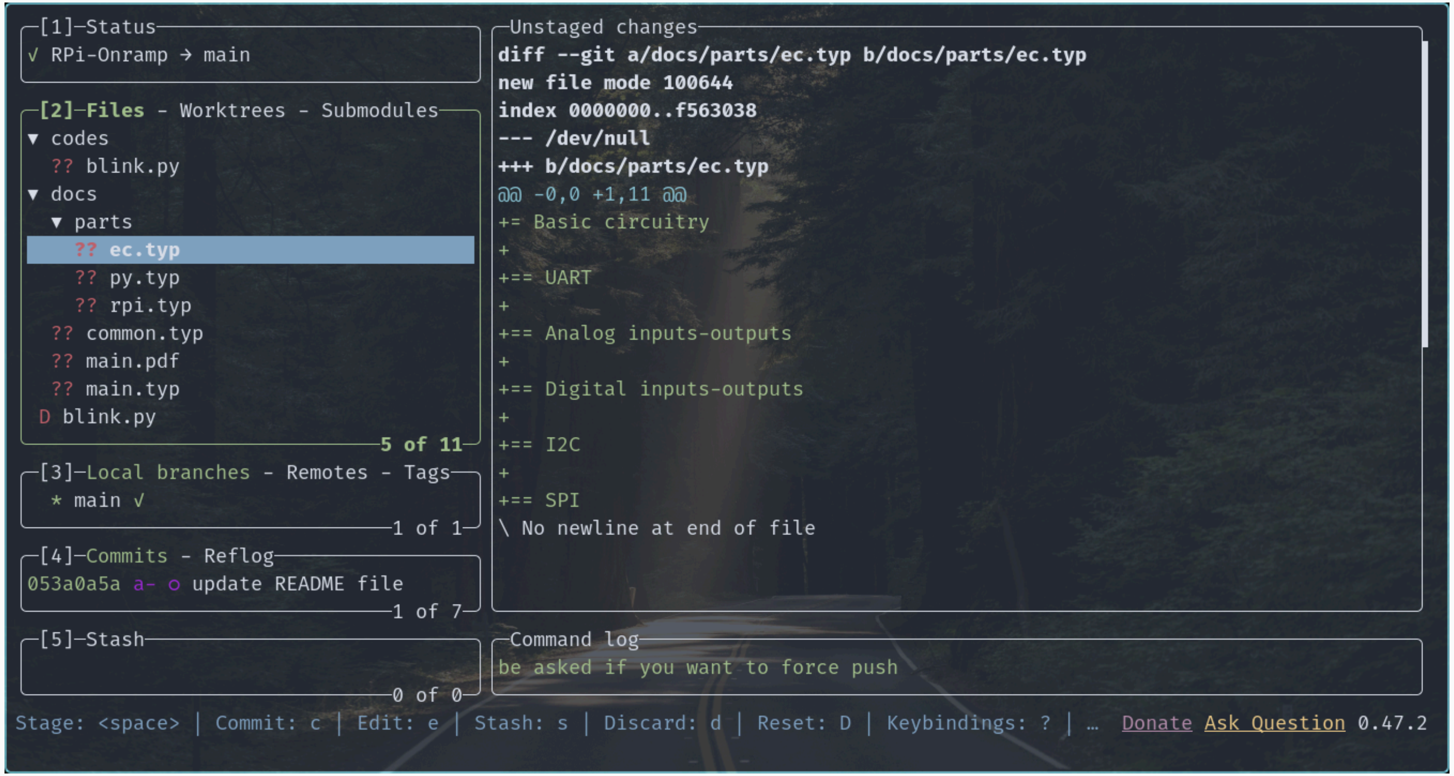
Lazygit

Lazygit is a terminal UI for Git that makes complex operations simple — stage individual lines, perform interactive rebases, and cherry-pick commits, all without memorising obscure flags. It runs directly in your terminal, so there is no need to switch between your editor and a separate GUI application. It is especially useful on the Raspberry Pi where you are often working entirely over SSH with no desktop environment.

To install it on Raspberry Pi OS:

```bash
sudo apt install lazygit
```

Launch it from inside any Git repository by simply running `lazygit`. Press ? at any time to view the full list of keybindings.



For full documentation and source code, see github.com/jesseduffield/lazygit.

Thank you for your attention

Bibliography

Bibliography

Axelson, J. (2007). *Serial Port Complete: COM Ports, USB Virtual COM Ports, and Ports for Embedded Systems* (2nd ed.). Lakeview Research.

Barnett, R. H., Cox, S., & O'Cull, L. (2006). *Embedded C Programming and the Atmel AVR* (2nd ed.). Thomson Delmar Learning.

Campbell, J. (1984). *The RS-232 Solution*. Sybex.

Cooley, J. W., & Tukey, J. W. (1965). *An Algorithm for the Machine Calculation of Complex Fourier Series* (Vol. 19, Issue 90, pp. 297–301). American Mathematical Society. https://doi.org/10.1090/S0025-5718-1965-0178586-1

EIA. (1987,). *EIA-232-D: Interface Between Data Terminal Equipment and Data Circuit-Terminating Equipment*.

Forouzan, B. A. (2006). *Data Communications and Networking* (4th ed.). McGraw-Hill.

Horowitz, P., & Hill, W. (2015). *The Art of Electronics* (3rd ed.). Cambridge University Press.

IEEE. (2022,). *IEEE Standard for Ethernet*. https://standards.ieee.org/ieee/802.3/10422/

Kugelstadt, T. (2003). *The I2C-Bus Specification*. Texas Instruments.

Bibliography

Ledin, J. (2021). *Modern Embedded Computing*. Packt Publishing.

Leens, F. (2009a). *An Introduction to I2C and SPI Protocols* (Vol. 12, Issue 1, pp. 8–13). https://doi.org/10.1109/MIM.2009.4762946

Leens, F. (2009b). An Introduction to I2C and SPI Protocols. *IEEE Instrumentation & Measurement Magazine*, *12*(1), 8–13. https://doi.org/10.1109/MIM.2009.4762946

Maier, A., Sharp, A., & Vagapov, Y. (2014). Comparative Analysis and Practical Implementation of the SPI Bus. *Proceedings of the Internet Technologies and Applications (ITA)*, Compare. https://doi.org/10.1109/ITechA.2014.6956339

Motorola. (2000). *SPI Block Guide* (V03.06). https://web.mit.edu/6.115/www/document/spi.pdf

NXP Semiconductors. (2021). *I2C-bus Specification and User Manual* (No. UM10204; 7th ed.). https://www.nxp.com/docs/en/user-guide/UM10204.pdf

Oppenheim, A. V., & Schafer, R. W. (1999). *Discrete-Time Signal Processing* (3rd ed.). Prentice Hall.

Raspberry Pi Ltd. (2024,). *Getting Started — Raspberry Pi Documentation*. https://www.raspberrypi.com/documentation/computers/getting-started.html

Bibliography

Stallings, W. (2014). *Data and Computer Communications* (10th ed.). Pearson.

Stoica, P., & Moses, R. L. (2005). *Spectral Analysis of Signals*. Prentice Hall.

Tanenbaum, A. S., & Wetherall, D. J. (2011). *Computer Networks* (5th ed.). Pearson Prentice Hall.

Thomas, G. B. (1949). A Self-Clocking Telegraph Signal. *Post Office Electrical Engineers' Journal*, *42*, 141–143.

TIA. (1997,). *Interface Between Data Terminal Equipment and Data Circuit-Terminating Equipment Employing Serial Binary Data Interchange* (Issue TIA-232-F).

Valvano, J. W. (2014). *Embedded Systems: Introduction to ARM Cortex-M Microcontrollers* (5th ed.). Self-published.